



evosoft Hungary Kft. Óbudai Egyetem

Nagy szoftverrendszerek
fejlesztése tantárgy
Tesztelés az ipari gyakorlatban



Restricted © evosoft Hungary Kft. 2026

www.evosoft.hu

Mi a tesztelés?

A tesztelés definíciói

Műszaki: Egy minőséget vizsgáló eljárás, módszer, amellyel egy anyag, termék, dolog adott szempont szerinti tulajdonságát, jellegét derítik fel, osztályozzák. (WikiSzótár.hu)

Műszaki: Egy hatást vizsgáló eljárás, módszer, amellyel egy (szélsőséges) körülmény, tényező fellépésének következményét vizsgálják egy helyzetre, állapotra, anyagra, tárgyra, személyre, élőlényre vonatkozóan (WikiSzótár.hu)

Szoftvertesztelés: A tesztelés egy rendszer vagy program kontrollált körülmények melletti futtatása, és az eredmények kiértékelése. A hagyományos megközelítés szerint a tesztelés célja az, hogy a fejlesztés során létrejövő hibákat minél korábban felfedezze, és ezzel csökkentse azok kijavításának költségeit. (Wikipedia)

Szoftvertesztelés: A tesztelés egy folyamat amely végig kíséri a terméket, az egész életciklusán át. Ez magába foglalja a követelmények megalkotásától, tervezés, kódolás, a termék megjelenése utáni utómunkálatokig mindent. A tesztelés alkalmas többek között meghatározni, hogy a követelmények teljesíthetők-e, igazolni, hogy a termék a kitűzött célnak megfelel, illetve annak minőségének javítására. (ISTQB)

Miért fontos a tesztelés?

„ A tesztelés azért szükséges mert mindnyájan emberek vagyunk és követünk el hibákat. Ezen hibák némelyike elhanyagolható, de néhány közülük drága és veszélyes. Akármit készítünk, azt ellenőriznünk kell, mert a dolgok mindig elromolhatnak – az emberek mindig hibáznak- ez az amiben a legjobbak vagyunk"

ISTQB Syllabus Foundation level



© evosoft Hungary Kft. | Restricted | Page 6

A tesztelés egy preventív tevékenység, vagyis megelőzés jelleggel hajtjuk végre. Komoly költségei vannak. Tekintsünk egy mai modern autót, ami legalább 8 légszákot tartalmaz. A légszákok emelik az autó árát és lehet, hogy sohasem fognak kinyílni. Azonban baleset esetén életet is menthetnek. A kisebb költségvetésű cégek nem, vagy csak korlátozott formában valósítják meg a tesztelést, amik károkat is okozhat a felhasználónak vagy magának a cégnek.



© evosoft Hungary Kft. | Restricted | Page 7

Elon Musk SpaceX Starship programja keretében több alkalommal is tesztelték a rakéta rendszer stabil működését. Különböző szintű tesztekert hajtottak végre annak érdekében, hogy a jövőbeli, embereket is szállító misszió során senkinek se eshessen baja. Látványos videókat lehetett látni arról, amikor a rakéta visszatérő fázisban hibásan landolt és emiatt felrobbant a teszteszköz. Szó szerint látható volt, ahogy dollármilliók válnak a hamu martalékává pillanatok alatt.

„Délután 16 óra óta nem működik a Facebook, ugyanis a felhasználók többségét kidobta a közösségi oldal weboldala és az alkalmazás egyaránt. Ezt követően a felhasználók nem tudnak visszalépni az oldalra.”
origo.hu

„A Downdetectorra több mint 300 ezer hibajelentés érkezett percek alatt.” origo.hu



© evosoft Hungary Kft. | Restricted | Page 8

2024. március 5-én őszén tapasztalható volt egy komolyabb globális probléma a Facebook szolgáltatásában. A hiba okáról mai napig nem nyilatkozott a Meta. De a hiba típusától függetlenül jól megmutatkozik, hogy milyen kihatással lehet az, ha nem végzünk megfelelő szintű tesztelést egy nagy, komplex rendszeren, ami sok felhasználót érint. Manapság a hétköznapiakban használt szoftverek esetén folyamatos frissítések formájában kapjuk a szoftver javításokat, biztonsági lyukak „foltozását” és az újabb funkciókat. Egy hibás működés így pillanatok alatt el tud jutni sok millió felhasználóhoz is, ha nem teszteljük megfelelően. Szerencsére ebben az esetben leginkább bosszúságot okozott az embereknek ez a probléma, ennél komolyabb hatása nem volt. (A mémgyártóknak viszont új táptalajt biztosított)

2024 július

„a CrowdStrike egyik hibás frissítése több millió ügyfél számítógépére került fel, ami globális technológiai leálláshoz vezetett. A frissítés megakasztotta az informatikai rendszereket: világszerte álltak le a légitársaságok, televíziók, bankok, kórházak és egyéb üzletek.” vg.hu



„A CrowdStrike okozta július 19-i informatikai leállása volt a világ legnagyobb informatikai kiesése, melynek következtében az amerikai Fortune 500 vállalat (a Microsoft kivételével) 5,4 milliárd dollár közvetlen pénzügyi veszteséget szenvedett el” vg.hu



© evosoft Hungary Kft. | Restricted | Page 9

A CrowdStrike egy amerikai kiberbiztonsági vállalat, amely a legmodernebb kiberbiztonsági megoldásokat kínálja az informatikai rendszerek számára. A világ egyik legnagyobb kiberbiztonsági szolgáltatója világszerte több ezer vállalkozásnak biztosít védelmi megoldásokat kibertámadás elleni. A cég elsősorban felhőalapú végpontvédelmi szoftverek, valamint kiberfenyegetések észlelésére és kezelésére specializálódott. Az általuk használt technológiák a mesterséges intelligencia és gépi tanulás révén képesek azonosítani és megakadályozni a kiberfenyegetéseket valós időben. Az iparágvezető kiberbiztonsági szoftver hibás frissítése világméretű leállásokkal járhat: a kék halál bankok, repterek, kórházak, biztosítók, vasúti rendszerek, de még futballklubok informatikai rendszereit is érintheti. A CrowdStrike általában nagy figyelmet kap a kiberbiztonsági szakmában, különösen olyan esetekben, amikor jelentős adatlopások vagy rendszerbehatolások történnek. A vállalat kulcsszerepet játszik az ilyen események gyors és hatékony kezelésében, valamint a jövőbeli hasonló támadások megelőzésében.

A kiberbiztonsági vállalat arról számolt be, hogy jelentős átalakításokat hajtanak végre a rendszereik frissítésében, hogy a jövőben ne fordulhasson elő hasonló leállás.

Ez tulajdonképpen azt jelenti, hogy a frissítések telepítését innentől az ügyfelekre bízzák, azok nem automatikusan kerülnek fel a rendszerekre. Így elkerülhető lehet, hogy legközelebb egy hibás frissítés tömeges leálláshoz vezessen.

„Két legfontosabb modelljében is szoftverhibát talált a Volkswagen csoport, átmenetileg leállt a 8-as Golfok és Skoda Octaviák kiszállítása. Sajtóhírek szerint mindkét modell vész hívó funkciójával van gond. A frissítésig a gyártósorról legördülő autók egyből raktárba kerülnek, a már átadott példányokra pedig visszahívás vár.”
Origo.hu



© evosoft Hungary Kft. | Restricted | Page 10

2020 tavaszán a Volkswagen-nél teljesen le kellett állítani a Golf-ok és a Skoda Octavia-k gyártását, mert szoftveres hibát észleltek a vész hívó gombok működtetése esetén. Egy gyártónak nagyon költségeket okoz egy gyártósor leállítása. Ebben az esetben azonban emberéleteket is veszélyeztető funkcionalitásról beszélünk, ami önmagában nem megengedhető, valamint egy ilyen eset bekövetkezése még komolyabb károkat tud okozni egy autógyártó számára.

BREAKING NEWS: 130 km/h-val a pályán

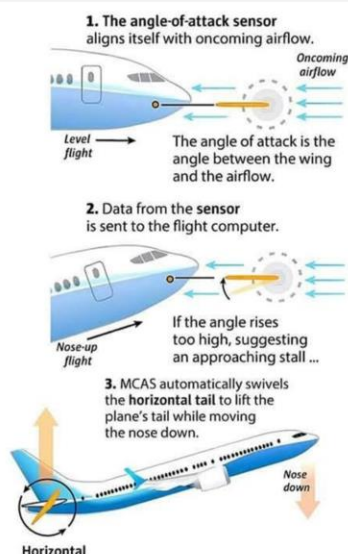
Durva kalandban volt része az egyesült államokbeli Irvine-ben élő Javier Rodrígueznek, amikor vadonatúj Tesla Model 3-asával az autópályán haladt. 2022 április 7-én, csütörtökön ugyanis meghibásodott autójában az Autopilot-rendszer, és állítása szerint a gombokon és a kapcsolókon kívül még a világítás sem működött – adta hírül az ABC7.



© evosoft Hungary Kft. | Restricted | Page 11

2022 tavaszán történt meg hogy egy Tesla 3-as szoftverrendszere 130 km/h-ás sebességnél az autópályán lefagyott. Ennek eredményeképp az autó több elektronikai eszköze sem reagált és mivel Autopilot módban üzemelt az autó, a gázpedálvezérlést sem lehetett visszavenni tőle. Szerencsére a fékrendszer független rendszeren működött, így a tulajdonosnak volt lehetősége lassítani az autót és végül megállni. A mai modern autókban a fejlesztő mérnököknek és informatikusoknak nagy felelősségük az implementált szoftverek fejlesztése során. Ahogy látható, egy szoftver hiba, akár emberéletet is veszélyeztethet. Ezért is látják el az ilyen kritikus rendszereket redundáns megoldásokkal. Ezek biztosítják a folyamatos felügyeletet és kontrollálhatóságot.

A tesztelés hiányának hatásai



Szoftverhiba miatt zuhant le két új típusú Boeing 737 MAX 8 gép

„Újabb szoftverhibát talált a Boeing vállalat a 737 MAX típusú gépein, amelyeket tavaly márciusban kivontak átmenetileg forgalomból két ilyen utasszállító katasztrófája után.”

Nepszava.hu



© evosoft Hungary Kft. | Restricted | Page 12

Egy vadonatúj típus, a Boeing-737MAX rövid időn belül történt két, összesen 346 halálos áldozatot követelő katasztrófája 2020-ban minden hírportál vezető híre lett. A típust világszerte a földre parancsolták, ami jelzi a probléma komolyságát. A halálos áldozatokon felül a légitársaságoknak a típus letiltása óriási veszteség, a Boeingnek pedig szintén hatalmas anyagi-és presztízsveszteség, hogy a világ legnépszerűbb repülőgéptípus-sorozatának legújabb tagja ilyen kontextusban került világszerte a figyelem középpontjába. A 2017-ben forgalomba állt 737MAX hajtóművei elődeihez, a 737NG-sorozathoz képest kissé feljebb és előrébb kerültek. Megváltozott a súlyponthelyzet, a sárkányszerkezet és a gép repülési tulajdonságai is. A nagyobb tolóerő a szárnyak alatt lógó hajtóművek miatt egyben azt is jelenti, hogy a repülőgépre egy, a gép orrát felemelni akaró nagyobb nyomatók hat, ami bizonyos körülmények között akár a gép átesését is okozhatja. Emiatt szükségessé vált egy olyan rendszer, ami a nagy tolóerő miatti, a gép orrát felemelő nyomatókat ezekben a helyzetekben kompenzálja. Ez az MCAS (Maneuvering Characteristic Augmentation System) – rendszer.

A 737MAX megújult hajtóművekkel, 7%-kal kisebb üzemeltetési költséggel, mint az elődei (boeing.com)

A Boeingon nagy volt a nyomás a légitársaságok felől, hogy a megváltozott repülési tulajdonságok ellenére se kelljen a 737NG-re kiképzett pilótáknak a 737MAX-hoz komolyabb típusátképzésen részt venniük. Ez is volt az egyik életre hívója ennek a rendszernek, hiszen automatikusan kompenzálta a repülési jellemzők egyikének a változását. Valószínű, hogy a pilóták nagy része nem is ismerte a rendszer működését.

Ha a gép átesésben van, vagy a rendszer ezt gondolja, akkor automatikusan – megszüntetni

akarván az átesés veszélyét – trimmelni kezdi a gépet, vagyis a gép orrát lefelé fordítja. A trimmelés a teljes vízszintes vezérsík segítségével történik, ennek a beállítási szöge változik ilyenkor. Ha a pilóták maguk trimmelik a gépet, akkor is ugyanazt a mechanizmust használva a vízszintes vezérsík beállítási szögét változtatják.

Az MCAS-rendszer működése – a vízszintes vezérsíkot állítja (b737.org.uk)

Az MCAS akkor lép működésbe, ha a fékszárnyak már behúzott helyzetben vannak, nincs bekapcsolva az autopilóta (vagyis a gépet kézzel vezetik) és azt érzékeli, hogy a repülőgép áteséshez közeli helyzetben van. Ez utóbbi pont az egyik kritikus része a rendszernek, ugyanis az esetleges átesés érzékelése egy, az orr-rész bal oldalán lévő állásszögadó jelei alapján történik.

Ethiopian Airlines Boeing B737 MAX8 típusú repülőgépe 50 kilométerre az Addis Ababa-i induló repülőtértől a felszállás közben 2019 március 10-én zuhant le, a fedélzeten tartózkodó 149 utas és a 8 fős személyzet életét veszítette.

Vélhetően ez a legsúlyosabb hiba a rendszerben, hiszen a repülőgépek, az egész repülőipar és a légiközlekedés minden fontos rendszerében megtalálható redundancia (sőt, többszörös redundancia) itt hiányzik. Az MCAS egyetlen állásszög-szenzor adatai alapján dönti el, hogy a gép átesésben van-e.

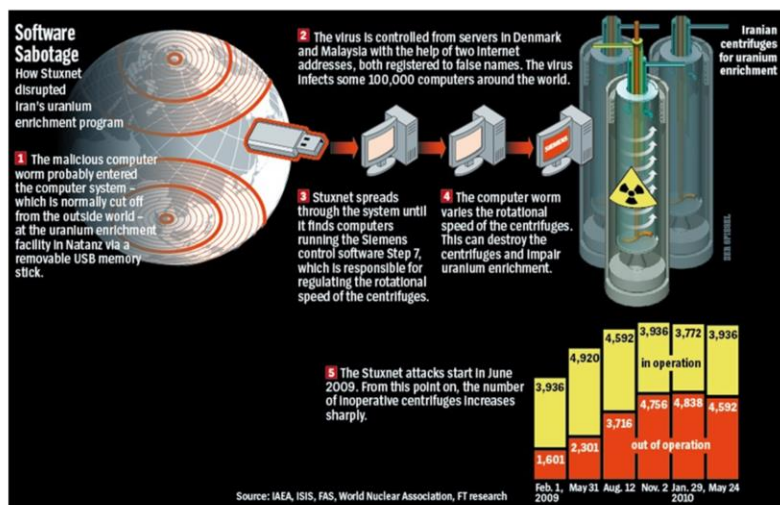
A bal oldali állásszög-szenzor, ami az etióp gépen hibásan működött. (CGTN.com)

A rendszer kb. 9 másodpercig működteti a trimmet, majd leáll. Ezután 5 másodpercig áll, majd ha úgy érzékeli, hogy az átesés még mindig fennáll, ismét aktiválódik és megint trimmelni kezd.

A katasztrófákat – az etióp gépet az előzetes jelentés alapján biztosan – vélhetően ez okozta. Az etióp gép esetében 74 fokos, teljesen abszurd állásszöget is „mért” a bal oldali szenzor és az MCAS-rendszer ennek alapján „gondolta” azt, hogy az egyébként normális üzemi tartományban repülő gép éppen átesésben van.

A pilótáknak nem volt esélyük kiiktatni a rendszert. Ez a rendszerhiba, a redundancia hiánya és a pilóták oktatásának hiánya okozta sok ember halálát.

Cyber támadás ipari létesítmény
ellen 2009-ben. (STUXNET)



© evosoft Hungary Kft. | Restricted | Page 13

A világszerte több mint százezer számítógépes rendszer megfertőző Stuxnetről eddig is tudni lehetett, hogy ipari létesítményeket támad, de az amerikai biztonsági cég munkatársainak legfrissebb kutatási eredményei szerint az iráni atomprogram lehetett a szerzőinek célpontja. A Symantec kutatói a júniusban Iránban felfedezett Stuxnet vírus kódját elemezve arra utaló nyomokat találtak, hogy a szoftver az ott urándúsításhoz használt centrifugák működésének sabotálására készülhetett - írja a Wired. A biztonsági cég nem jelentette ki ezt egyértelműen, de munkatársai számos olyan részletet feltártak a szoftverrel kapcsolatban, ami erre utal, beleértve az általa sabotált berendezések pontos típusát, amelyek egyértelműen az iráni atomprogramhoz kötődnek.

Átállítja az urándúsító centrifugákat

A világszerte százezer számítógépet megfertőző vírusról már felfedezése után kiderült, hogy a Siemens SCADA nevű ipari szoftverének sebezhetőségeit használja ki. A kutatók eredetileg csak azt feltételezték, hogy a Stuxnet adatokat fog el vezérlőszoftvertől, de nem ismerték a pontos működését. Mostanra azonban kiderült, hogy a szoftver nem is egyszerűen a SCADA-ra kapcsolt gyártósorok, szivattyúk vagy erőmű-berendezések elrontására képes, hanem pontosan meghatározott típusú urándúsító centrifugákat keres rajta, és egészen kifinomult módszerekkel avatkozik be azok működésébe.

A Stuxnet, ha sikerült megfertőznie egy SCADA rendszert, átvizsgálja a hálózatot, és csak akkor lép működésbe, ha ott legalább 33 darab, az iráni Fararo Paya, vagy a finn Vacon által gyártott frekvenciaváltó berendezést talál. Ezeket az urándúsító centrifugák motorjának vezérlésére használják, legalábbis azon a magas frekvencián, amit a vírus keres. Ha szoftver megtalálja az említett alkatrészeket, akkor átállítja azok forgási sebességét, úgy lassítva-gyorsítva azokat, hogy végeredményben lerontsa az általuk előállított uránium minőségét.

Akár incidenst is okozhatott, bár rejtőzködni kell

Ahhoz, hogy ezt a fajta szabotázszt sikeresen végrehajtsa, a vírusnak hosszú időn át észrevétlenül kell dolgoznia a rendszerben, vagyis nem teheti tönkre azokat, amivel el is árulná magát. A Wired szerint a Stuxnet már 2009 elején fertőzni kezdhették az iráni atomlétesítményekben, és akár a vírus is okozhatta az év közepén Natanzban bekövetkezett súlyos incidenst is, amelyet követően az országban működő urándúsító centrifugák száma máig ismeretlen okból 4700-ról 3900-ra csökkent. (Az iráni atomprogramról a katonai célú tudományos kutatásokkal foglalkozó tudományos szervezet, a FAS, vagyis Federation of American Scientists közölt statisztikákat, de az incidens részletei előttük is rejtve maradtak.)

A Symantec kutatói szerint ilyen összetett és célzott szabotázszt végző szoftver létrehozására csak egy "anyagi erőforrásokban bővelkedő nemzetállam" által megbízott programozók lehetnek képesek. A vírust egy titokzatos belorusz biztonsági cég, a VirusBlokAda fedezte fel egyik iráni ügyfele gépén, és ezt követően német biztonsági szakértők is megvizsgálták, akik akkor azt a lehetőséget sem zárták ki, hogy az fel tudna robbantani egy atomerőművet. Forrás: <https://www.origo.hu/techbazis/20101116-az-urandusitok-ellen-irtak-a-stuxnet-virust-a-symantec-szerint.html>

A tesztelés célja

Szoftverminőség biztosítása

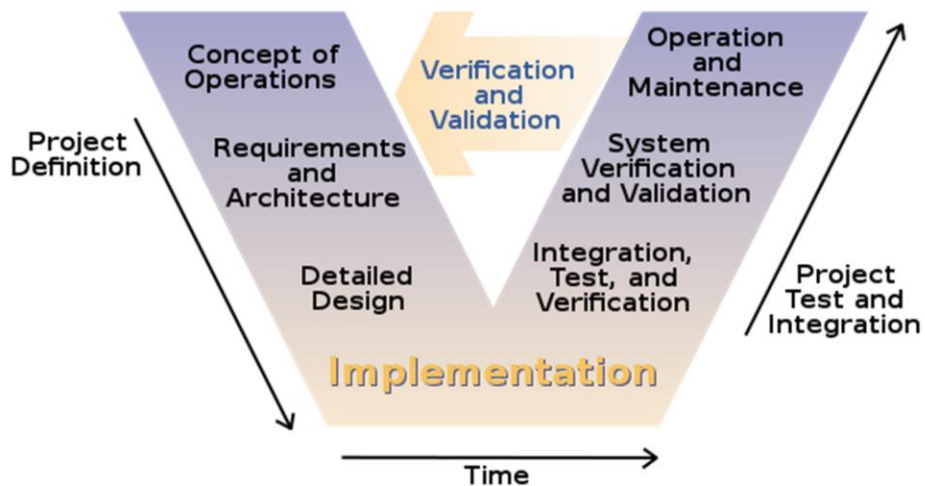
Szoftver hibák lokalizálása, javítása

Biztonságkritikus hibák felderítése
szabványok mentén

Információ a döntéshozáshoz (pl.
release/ no release)



Tesztelés = verifikáció + validáció

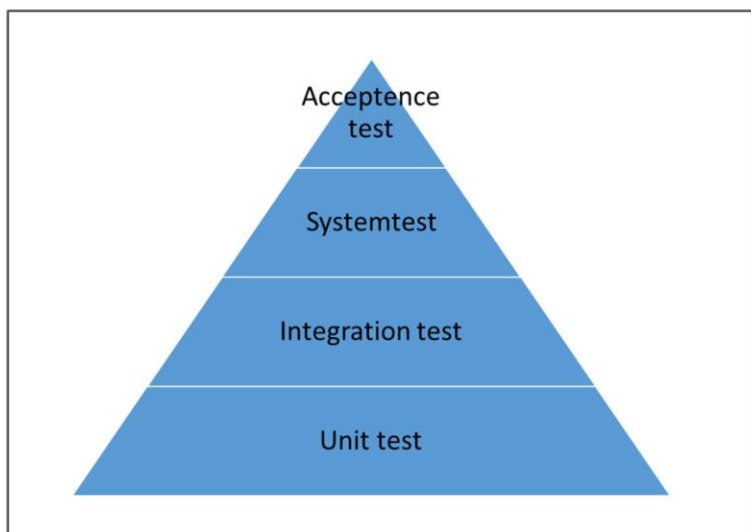


© evosoft Hungary Kft. | Restricted | Page 15

Tesztelés = verifikáció + validáció

Verifikáció: jól építettük-e fel a rendszert?

Validáció: jó rendszert építettük-e fel?



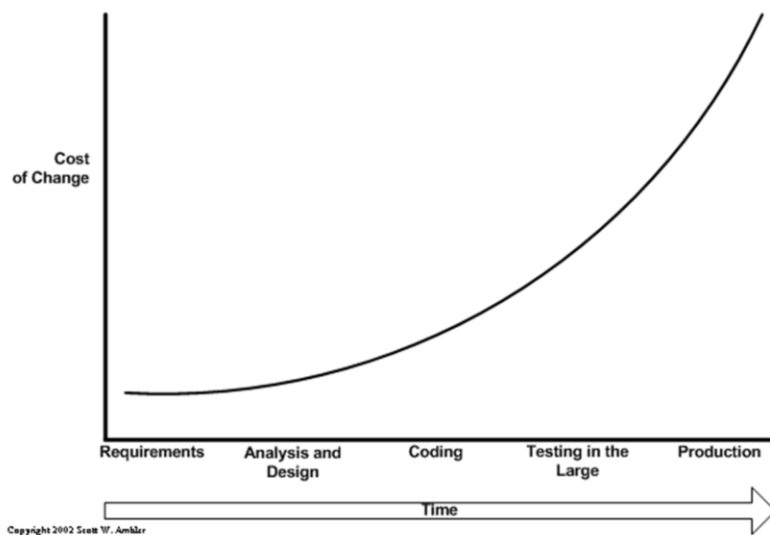
© evosoft Hungary Kft. | Restricted | Page 16

- Unit teszt: A rendszer önálló részeit teszteli a forráskód ismeretében (fehér dobozos tesztelés)
- Integrációs teszt: Az integrációs teszt kettő vagy több komponens együttműködési tesztje az interface-eken keresztül
- Rendszer teszt: A kész szoftverterméket teszteli (fekete dobozos tesztelés), hogy megfelel-e a követelmény, ill. funkcionális specifikációnak.
- Átvételi teszt: Az egész rendszert teszteli üzleti szemmel, de ezt már a végfelhasználók végzik.

Azért piramis alakú a tesztpiramis, mert a végrehajtott tesztek számát szimbolizálják az egyes szintek. Cél az, hogy már korai fázisban minél több teszt végre legyen hajtva. Érdeemes

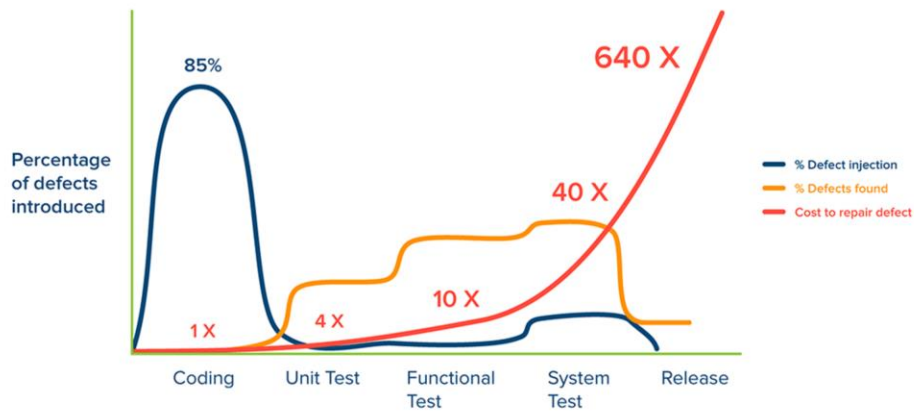
elemezni az egyes szervezeteknél, hogy néz ki a tesztpiramis a valóságban. Amennyiben eltorzult, vagy fejtetőre állított piramis alakot ad, ott érdemes a tesztstratégián változtatni.

Miért fontos a korai tesztelés?



Minél később derül ki a hiba, annál drágább javítani

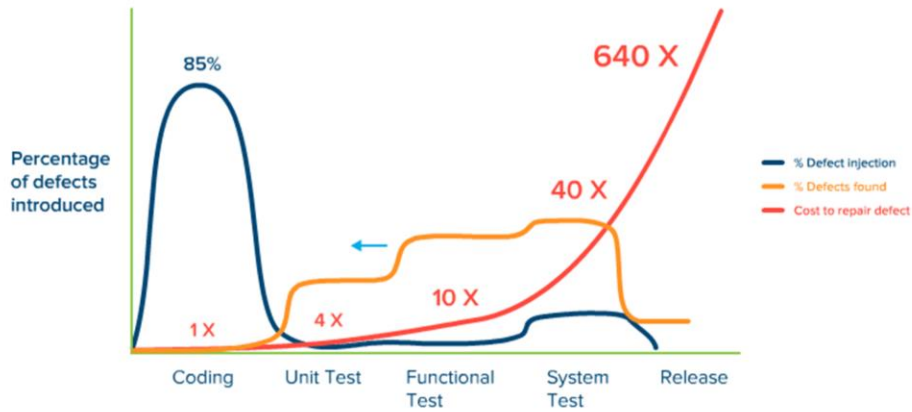
Miért fontos a korai tesztelés? Shift- left (korai tesztelés)



Jones, Capers. *Applied Software Measurement: Global Analysis of Productivity and Quality*.

<https://www.parasoft.com/blog/what-is-shift-left-testing/>

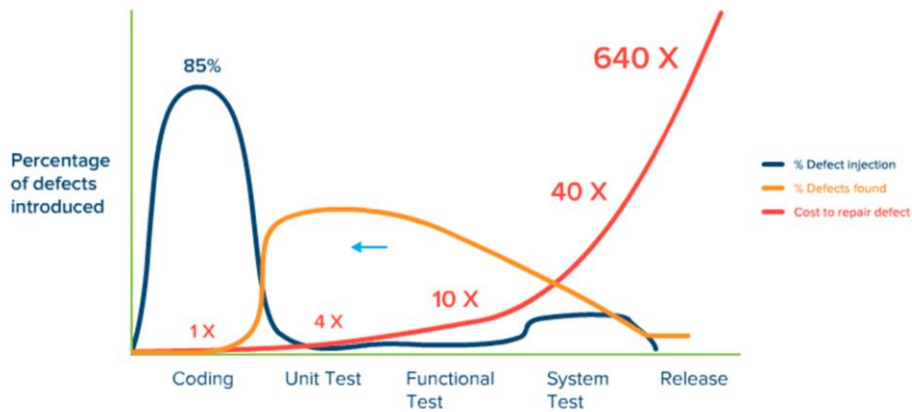
Miért fontos a korai tesztelés? Shift- left (korai tesztelés)



Jones, Capers. *Applied Software Measurement: Global Analysis of Productivity and Quality*.

<https://www.parasoft.com/blog/what-is-shift-left-testing/>

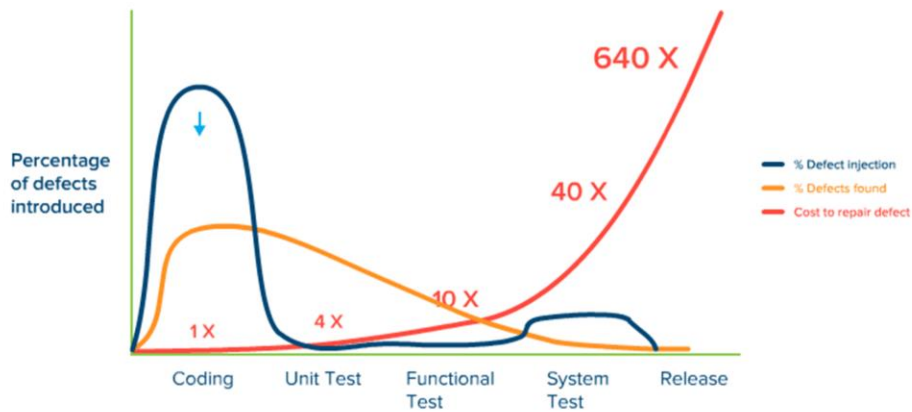
Miért fontos a korai tesztelés? Shift- left (korai tesztelés)



Jones, Capers. *Applied Software Measurement: Global Analysis of Productivity and Quality*.

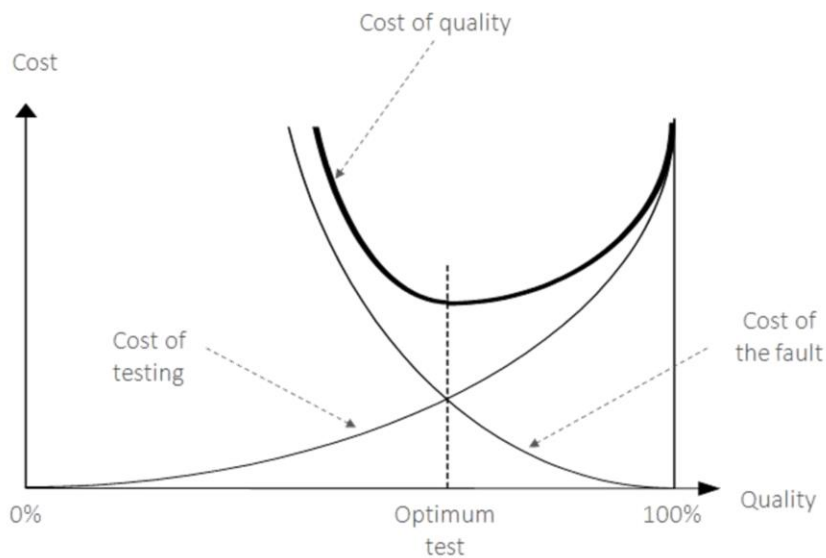
<https://www.parasoft.com/blog/what-is-shift-left-testing/>

Miért fontos a korai tesztelés? Shift- left (korai tesztelés)



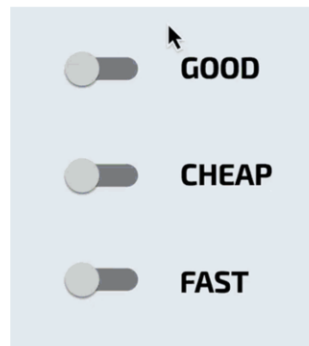
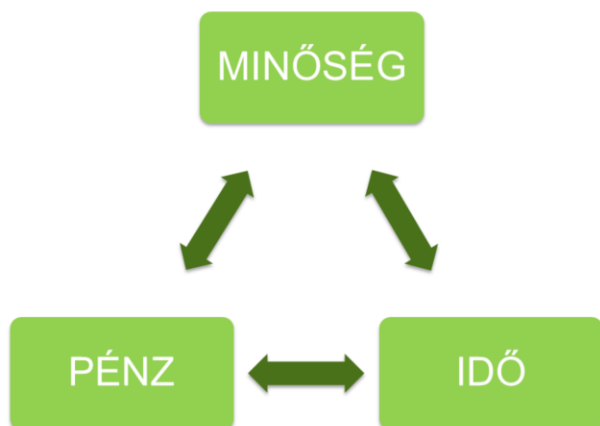
Jones, Capers. *Applied Software Measurement: Global Analysis of Productivity and Quality*.

<https://www.parasoft.com/blog/what-is-shift-left-testing/>



© evosoft Hungary Kft. | Restricted | Page 22

A termékminőségünk költségeit a tesztelés költségei és a hibák javításának költségei határozzák meg. Ennek van egy optimum pontja, ahol a két költségtényező összege a legalacsonyabb. Üzletileg ezt a pontot keressük és ezzel határozzuk meg a tesztelésre fordított erőforrást.



Valamiből engednünk kell!
Ezért kell tervezni a tesztelési alapelvek mentén!

01 TESZTELÉS MÁR A KORAI FÁZISBAN

02 NEM LEHETSÉGES KIMERÍTŐ TESZT

03 A TESZTELÉS A HIBÁK JELENLÉTÉT JELZI

04 HIBÁK CSOPORTOSULÁSA

05 A TESZTELÉS FÜGG A KONTEXTUSTÓL

06 HIBÁTLAN RENDSZER TÉVESZMÉJE

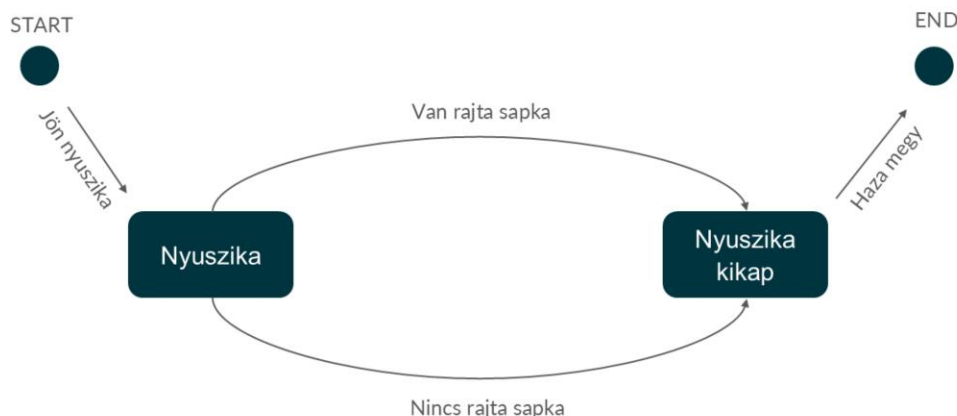
Érdemes már a korai fázisokban tesztelni, hiszen annál olcsóbb lesz a hibajavítás. Sohasem fogjuk tudni a szoftverünk működése során fellépő összes scenáriót letesztelni már egy egyszerű termék esetén sem.

Dijkstra: A tesztelés a hibák jelenlétét, és nem a hibamentességet tudja kimutatni, de azokat legalább tudjuk javítani.

Mivel szoftver programjainkat strukturáltan építjük fel, ezért a jelentkező hibák is csoportosulnak egy-egy komponens vagy funkcionalitás köré. Ezért érdemes a gyökérokot felkutatni már tesztelés során is, ez segíti a hatékony javítást.

Tesztelés során nem mindig van lehetőségünk valós adatokkal és scenáriókkal dolgozni. Ez megvezetheti a tesztelés folyamatát. Ezért törekedni kell a valósághoz leginkább hasonlító tesztkörnyezet felépítésére, így a release-t követő használatból nem alakulhat ki olyan kontextus, amire nem lett letesztelve a rendszer.

Hibátlan rendszer, szoftver nincs. Még akkor sem a tesztek ezt mutatják. Ilyenkor el kell gondolkodni, hogy a tesztekben lehet a hiba.



Egy teszteset lefutása leginkább egy állapotátmenet gráffal írható le. A kezdő állapotból átlépünk következő állapotokba. Az állapotok egymásutánja fogja kiadni a teszt esetet. Egy adott teszteset azonos feltételek és lefutás mellett mindig ugyanazt az eredményt fogja adni. A fenti állapotátmenet gráf két teszt esettel fedhető le:

Jött nyuszika -> Megvizsgálásra került, van-e rajta sapka -> Van -> Nyuszika kikapott -> Nyuszika hazament.

Jött nyuszika -> Megvizsgálásra került, van-e rajta sapka -> Nincs -> Nyuszika kikapott -> Nyuszika hazament.

Amennyiben mindkét előző esetben az állapotok végrehajtottak, akkor teszt eseteink sikeresek voltak. Ha valamelyik ponton nem jó állapotba került a programfutás, akkor hibásként kerül megjelölésre a teszt futás.

- **Specifikáció alapú technikák:**

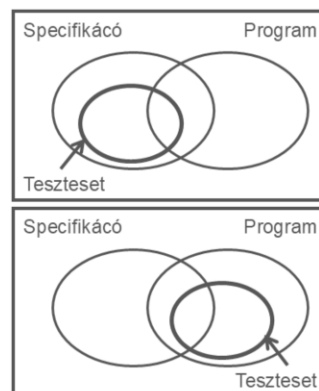
- Specifikációból vezetnek le a teszteseteket
- A rendszer mint „fekete doboz” adott

- **Struktúra alapú technikák:**

- Forráskódból vezetnek le a teszteseteket
- Fehér doboz technika

- **Gyakorlat, tapasztalat alapú technikák**

- Kevés az idő, hiányos dokumentáció => korábban megszerzett tapasztalat



Csoportosítás egy meghatározott tesztelési cél szerint

- Funkcionális minőségi jellemző (specifikált funkciók az alapja)
- Nem-funkcionális minőségi jellemző (megbízhatóság, teljesítmény)
- Változtatások ellenőrzése

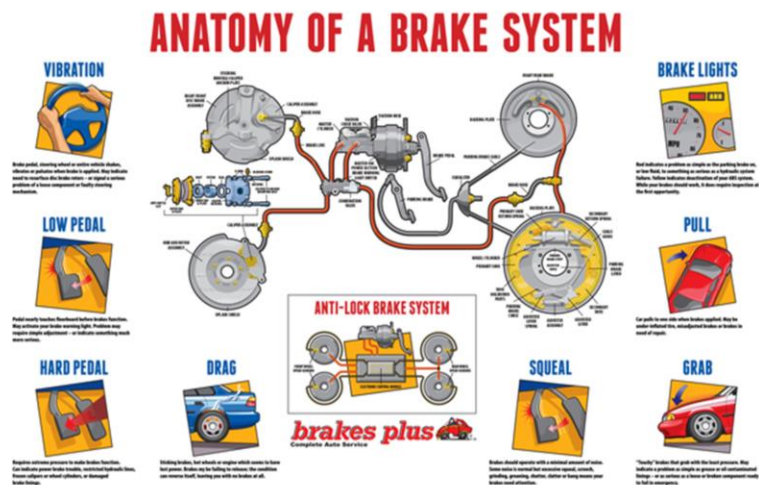


Funkcionális teszt

Nem funkcionális (NFR) teszt

Regressziós teszt

Milyen funkcionális tesztek
lehetne definiálni egy autó fék-
rendszeréhez?



© evosoft Hungary Kft. | Restricted | Page 28

Ha megnyomjuk a fékpedált -> az autó lassulni kezd
Ha teljesen lenyomjuk a fékpedált -> az autó kerekei állóra fékeződnek (kivéve ha ABS
rendszerrel van felszerelve)
Stb.

01 TELJESÍTMÉNYTESZT

- Mit akarunk mérni?
- Szimulálni kell a terhelést.
- Skálázhatóság kiderítése.
- Felfedni a gyenge teljesítmény okát.

02 TERHELÉSI TESZT (STRESS TEST)

- CPU és memória felhasználás.
- Adatbázis diagnosztikai eszköz.
- Naplózás.
- Profil készítő eszköz.

03 STABILITÁS TESZT

- Hol fogják használni?
- Long running tesztek.

04 HASZNÁLHATÓSÁGI TESZT

- Ergonómiai problémák felfedése.
- Termékkel való elégedettség.
- Interaktív megoldás.
- Video rögzítés.

1. Pl. játék szoftverek teljesítmény igénye (videókártya)
2. SMS küldős TV műsorok esetén sokszor volt tapasztalható, hogy leállt a rendszer túlterhelés miatt
3. Orvosi eszközök szoftverei: lélegeztető gép. Autóipari szoftverek
4. Negatív példa: állami szervek szoftverei

Mit is vizsgálunk?

A működést különböző software és hardware környezetben(OS, browser).

Közös környezetben való együttműködés más programokkal(erőforrás).

Mikor használhatjuk még?
Verzióváltáskor!



Siri



Google Assistant



Hey Cortana



Manapság az egyes gyártóknak piaci érdekük, hogy termékeik kompatibilisek legyen más gyártók termékeivel, mivel a felhasználók szeretnék egyszerűen, gyorsan más szolgáltatásokat nyújtó termékeikkel használni azokat. Pl.: különböző asszisztens szolgáltatások: Siri, Google Assistant, Alexa, stb. ezeket az együttműködések is tesztelni kell, mielőtt piacra kerül a termék. De a kompatibilitás tesztelés akkor is előkerül, amikor egy gyártó termékéből egy új verzió keletkezik, viszont egy korábbi verzióban készített felhasználói állományt szeretnék kezelni benne. Pl. MS Excel különböző verzió közötti kompatibilitás.



A biztonsági tesztelés egy olyan szoftvertesztelési típus, amely a rendszer sebezhetőségének felismerésére, valamint annak információk és erőforrások védelmének biztosítására szolgál a lehetséges betolakodókkal szemben.

© evosoft Hungary Kft. | Restricted | Page 31

A cyber security terület és a hozzá kapcsolódó teszt tevékenység nagy hangsúlyt kapott az elmúlt években. Nagyon sok információt tárolunk tőlünk távol lévő, felhő alapú adattárolókon. Ezek az adatok lehetnek személyes tartalmak, de cégek belső információ is. Sok értéket képviselnek, de nem csak nekünk, hanem versenytársainknak is. Pont ezért kell védenünk ezeket. Már termékfejlesztés korai szakaszában szakértőket vonunk be, akik felhívják a figyelmet a hiányosságokra és segítik a fejlesztést. Ezt kiegészítve a tesztelési piramis különböző szintjein pedig kimondottan security fókuszú teszt tevékenységet hajtunk végre, amik feltárják a rendszer sebezhetőségét.

1. Sebezhetőség vizsgálata

Ezt automatikus szoftverrel hajtják végre, hogy megvizsgálják a rendszert a biztonsági rés ismert aláírásait illetően.

2. Biztonsági szkennelés

Ez magában foglalja a hálózat és a rendszer gyengeségeinek azonosítását, és alternatívákat kínál az ilyen veszélyek csökkentésére. A kézi és az automatikus szkenneléshez ez a szkennelés elvégezhető.

3. Behatolásteszt

Ez a teszt egy rosszindulatú hacker támadást szimulálja. Ez a vizsgálat magában foglalja egy speciális rendszer elemzését a belső hackelés esetleges sebezhetőségének felismerésére.

4. Kockázatértékelés

Ez a teszt magában foglalja a vállalat által megfigyelt biztonsági veszélyek elemzését. A kockázatok alacsony, közepes és magas besorolással rendelkeznek. Ez a teszt kockázatcsökkentési ellenőrzéseket és intézkedéseket javasol.

5. Biztonsági ellenőrzés

Az ellenőrzés online, vonalon keresztül, kódellenőrzésekkel és a biztonsági hibák operációs rendszereivel is elvégezhető.

6. Etikus csapkodás

Az etikus hackelés nem ugyanaz, mint a rosszindulatú hackelés. Az etikus hackelés célja a szervezeti felépítés biztonsági hiányosságainak azonosítása.

7. Testtartás-értékelés

Ez egyesíti a biztonsági átvizsgálást, a kockázatértékelést és az etikus hackelést, hogy megmutassa a szervezet általános biztonsági helyzetét.

Teszt típusok

Regressziós teszt

Ismételt végrehajtás

Minden szinten használható: funkcionális, nem funkcionális és strukturális tesztek

Manuális vs. automatikus teszt

Regressziós tesztcsomag

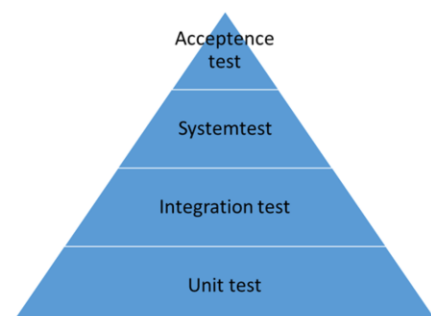
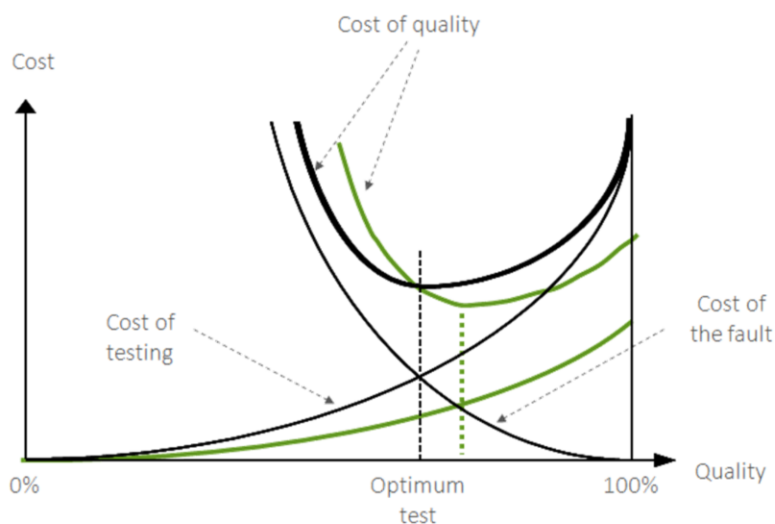
Folyamatos karbantartás, kibővítés

Mi teendő akkor, ha túl nagy a regressziós tesztcsomag?



<https://anarsolutions.com/regression-testing/>

A tesztelés költségei



© evosoft Hungary Kft. | Restricted | Page 33

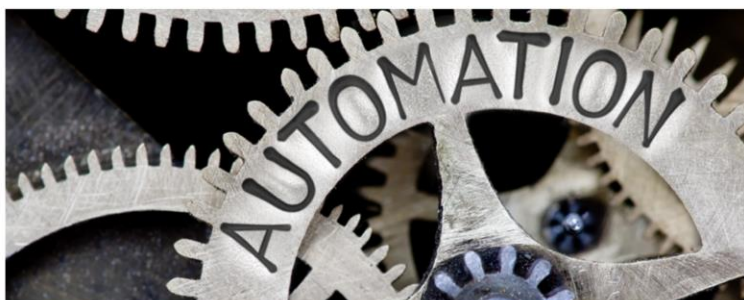
Korábban volt róla szó, hogy a cégek a tesztelést igyekeznek az minőség költségének optimum pontján tartani. Ez meghatároz egy adott minőségi szintet a szoftverben. Viszont vannak módszerek, amikkel a tesztelés költsége csökkenthető, így ez az optimum pont eltolható a jobb minőség irányába. Ennek egyik forrása az ismétlődően végrehajtott regressziós tesztek, ahol minden futtatás alkalmával ugyanazt az eredményt várjuk el. Ha valamit többször, ugyanolyan formában kell végrehajtani, azt érdemes automatizálni.

Előnyök:

- Csökken az ismétlődő munka
- Jobb konzisztencia és megismételhetőség
- Objektív elemzés
- Információbőség

Kockázatok:

- Bevezetés
- Karbantartás
- Együttműködés
- Üzleti kockázat
- Support
- Upgrade



Szükséges-e? Alapkérdések:

- Hányszor kell futtatni teszteket?
- Meddig fogjuk ezeket a teszteket használni?
- A jövőben mennyi új hibát fogunk ezekkel a tesztekkel megtalálni?

© evosoft Hungary Kft. | Restricted | Page 34

Az automatizált tesztek a nap 24 órájában tudnak futni a tesztpiramis különböző szintjein. Egy jól megírt teszt program, nem hibázik. Ugyanarra az inputra, mindig ugyanazt az eredményt fogja adni, amennyiben a környezet nem változik. Folyamatosan dokumentálható a tesztfutás állapota, eredményei, így hamar visszakéreshető, ha hiba történik a teszt során. Azonban egy ilyen rendszert fel kell építeni és üzemben kell tartani. Ennek ellenére gazdaságosabban üzemeltethető, mint ha manuálisan hajtánánk végre. A bevezetéskor érdemes megvizsgálni, milyen szoftvert fogunk vele tesztelni, milyen szintjén a teszt piramisnak, hányak fogják használni, milyen távon szeretnénk használni, van-e megfelelő támogatottsága egy külső cég által fejlesztett tesztkörnyezetnek, van-e megfelelő számú szakemberünk.

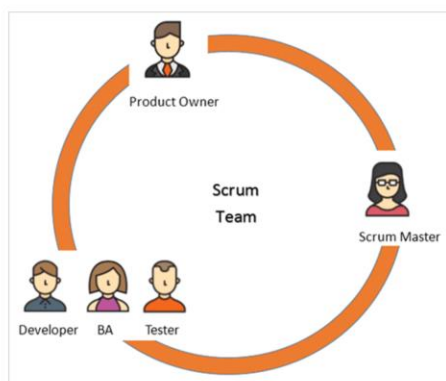
- Tesztelési tevékenység mérésére szolgál.
- MÉRŐSZÁMOK
 - Specifikáció alapú technikák
 - Struktúra alapú technikák
 - Általában 0 és 1 közé eső értékek
 - 100% lefedettség kérdése



- Teszt lefedettség $= \frac{\text{mért lefedettségi elemek száma}}{\text{összes elem száma}} \times 100\%$
- Utasítás lefedettség $= \frac{\text{mért utasítások száma}}{\text{összes utasítás száma}} \times 100\%$
- Ág (döntési) lefedettség $= \frac{\text{mért döntési kimenetek száma}}{\text{összes döntési kimenetek száma}} \times 100\%$
- Út lefedettség $= \frac{\text{mért utak száma}}{\text{összes utak száma}} \times 100\%$

A tesztlefedettség segít nekünk következtetni a minőségre és segít mérni a teszt tevékenységünket. Értelmezhető specifikáció alapú technikáknál és struktúra alapú technikáknál is. Azonban a specifikáció alapú technikáknál nehéz meghatározni az összes esetszámot, így hozzávetőleges értéket fogunk kapni. A struktúra alapúak esetén egyszerűbb a helyzet, mert azonosítható, hogy a programkódban hány utasítás, döntési ág, és bejárt út található. Az ezekre megírt tesztek és azok aránya összes lehetséges esetszámmal fogja meghatározni a lefedettségi értéket. Vannak olyan területek, ahol szabvány írja elő, hogy milyen tesztlefedettségi szintet kell elérni egy termék kapcsán, hogy az piacra kerüljön. Pl. autóiipari szoftverek.





© evosoft Hungary Kft. | Restricted | Page 37

A tesztelő szerves részét képezni egy scrum csapatnak. Már a tervezési (grooming) fázisban tud a termék használatával kapcsolatban hasznos információkkal szolgálni, így legkorábbi pontban a követelményspecifikációkor megjelenik a tesztelési szemlélet. A sprint során pedig különböző teszt szinteken tudja biztosítani a minőségi termékfejlesztést és sprint végi szállítást.

A szoftvertesztelői tevékenység áttekintése

- Kiemeltfontosságú egy fejlesztői csapatban
- Nem csak manuális tevékenység, hanem programozói skilleket is igényel
- A termék magasszintű ismerete jellemzi
- Komoly piaci értéket képvisel



© evosoft Hungary Kft. | Restricted | Page 38

<https://www.designveloper.com/blog/effective-software-tester-skills/>

ChatGPT és a tesztelés

AI Kérlek egy programkódot, ami leteszteli az Óbudai Egyetem Neumann János Informatikai Karának honlapján az "Elérhetőségek" menüpontjának funkcionális működését és amit le tudok futtatni Selenium környezetben!

Természetesen, itt találsz egy Python kódot, amely leteszteli az Óbudai Egyetem Neumann János Informatikai Karának honlapján az "Elérhetőségek" menüpontjának működését és Selenium-ban futtatható:

```
python
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC

# Selenium driver inicializálása
driver = webdriver.Chrome()

# Oldal betöltése
driver.get("https://www.uni-obuda.hu/")

# Az "Elérhetőségek" linkre kattintás
elkerhetosegek_link = driver.find_element_by_link_text("Elérhetőségek")
elkerhetosegek_link.click()

# Várakozás, amíg betöltődnek az elérhetőségek
wait = WebDriverWait(driver, 10)
wait.until(EC.visibility_of_element_located((By.CSS_SELECTOR, "#main-content")))

# Az elérhetőségek címének ellenőrzése
elkerhetosegek_cim = driver.find_element_by_css_selector("#main-content h1")
assert elkerhetosegek_cim.text == "Elérhetőségek"

# Selenium driver leállítása
driver.quit()
```

Ez a kód megnyitja a Chrome böngészőt, betölti az Óbudai Egyetem Neumann János Informatikai Karának honlapját, majd rákattint az "Elérhetőségek" linkre. Végül ellenőrzi, hogy a betöltött oldal címe "Elérh" Regenerate response megfelelő, akkor egy AssertionError lesz dobva. Ha a cím nem az, a Selenium webes chatbotok analízis hogy

AI Generálj 2 teszt esetet egy számlázó szoftverhez!



Teszt eset 1:

Cél: Ellenőrizni a számlázó szoftver helyes működését a számlák kiállításakor.

1. Adott: Ügyfél adatai, termék neve, mennyisége és ára.
2. Végrehajtás: Kiállítunk egy számlát az adott ügyfélnek a megadott termékekkel és az árukkal.
3. Ellenőrzés: Ellenőrizzük, hogy a számla tartalmazza az összes megadott információt, helyesen számolja ki az árakat, és az összes adókulcsot és végösszeget helyesen számolja ki.

Teszt eset 2:

Cél: Ellenőrizni a számlázó szoftver helyes működését az adatbázis frissítésekor.

1. Adott: Adatbázisban szereplő ügyfelek és termékek, új ügyfél és termék adatok.
2. Végrehajtás: Frissítjük az adatbázist az új ügyfél és termék adatokkal, majd kiállítunk egy számlát az új ügyfélnek az új termékkel.
3. Ellenőrzés: Ellenőrizzük, hogy az új ügyfél és termék adatok sikeresen felkerültek az adatbázisba, és a számla helyesen számolja ki az árakat, az adókulcsokat és a végösszeget a frissített adatok alapján.

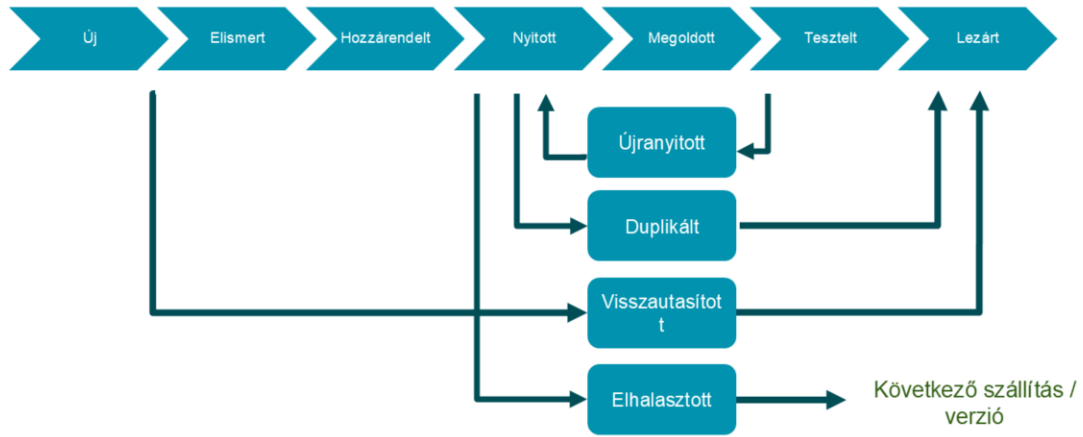
Az általános célú AI-ok elterjedése a tesztelési tevékenységet is tudja támogatni. Képes teszt esetek generálására és olyan automatizált tesztek kódjának megírására, ami valamilyen tesztfutató környezetben képes működni. Minden esetben azonban szükséges egy szakember, aki ellenőrzi az eredményt, hiszen az AI-tól kapott megoldás csak annyira jó, amennyire a problémakörnyezet jól körül határolt és a számára szükséges információk a megfelelő szinten rendelkezésre állnak.

- Hol követjük a hibákat?
- Hibamanagement feladatai:
 - hibára utaló tünetek meghatározása
 - hiba izolálása
 - hiba kijavítása és megoldás tesztelése
 - hibajelenség és a megoldás dokumentálása

Hibakövető rendszer



- Hibaleírás új hiba esetén:
 - Cím
 - Súlyosság: Akadály / Összeomlás / Súlyos / Nem súlyos / Nyilvánvaló / Szöveg
 - Részletes leírás
 - Verzió
 - File
 - Reprodukálhatóság: Mindig / Véletlenszerű / Nem ismételhető



- Agilis módszertan: Test Driven Development
- Linkek:
 - <http://www.istqb.org/> (*International Software Testing Qualifications Board*)
 - <http://www.hstqb.com> (*Hungarian Software Testing Qualifications Board*)

Contact

Attila Garamvölgyi

Domain manager

Mobile +36 (30) 193-2230

E-mail attila.garamvolgyi@evosoft.com

Jelen alkotás szerzői jogi védelem alatt áll. A felhasználásához szükséges engedélyért keresse: evosoft Hungary Kft. H-1117 Budapest, Magyar tudósok körútja 11., Tel: +36 (1) 38-16400



Your powerful partner

www.evosoft.com